

TTK4190 - GUIDANCE, NAVIGATION AND CONTROL OF MARINE
CRAFT, AIRCRAFT AND DRONES

Assignment 2: Part 2

Heading Autopilot & Speed Control

Authors:

Audun Bagøyen Hustad
Bendik Letting Skari
Fredrik Siem Taklo
William Hammer

29th October 2025

Table of Contents

1	Environmental Disturbances	1
1a	2D irrotational currents in surge and sway	1
1b	Simulation results	1
1c	Adding wind moments	2
2	Heading Autopilot	3
2a	Linearizing Coriolis forces	3
2b	Transfer function	3
2c	Nomoto model	3
2d	PID Controller	4
2e	Controller performance	7
3	Propeller Revolution and Speed Control	10
3a	Propeller thrust and torque coefficients	10
3b	Torque to speed	10
3c	Propulsion system code	10
3d	Cruise speed	11
3e	Desired thrust, speed & simulation	12
3f	Optional Improvements	13

1 Environmental Disturbances

1a 2D irrotational currents in surge and sway

Currents added to main.m

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% Part 2, 1a) 2D irrotational current  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
Vc = 1; % (m/s)  
betaVc = 45*pi/180; % 45 deg CW from N -> NE (rad)  
  
uc = Vc * cos(betaVc - x(6)); % BODY Surge current (m/s)  
vc = Vc * sin(betaVc - x(6)); % BODY Sway current (m/s)  
nu_c = [ uc vc 0 ]';
```

1b Simulation results

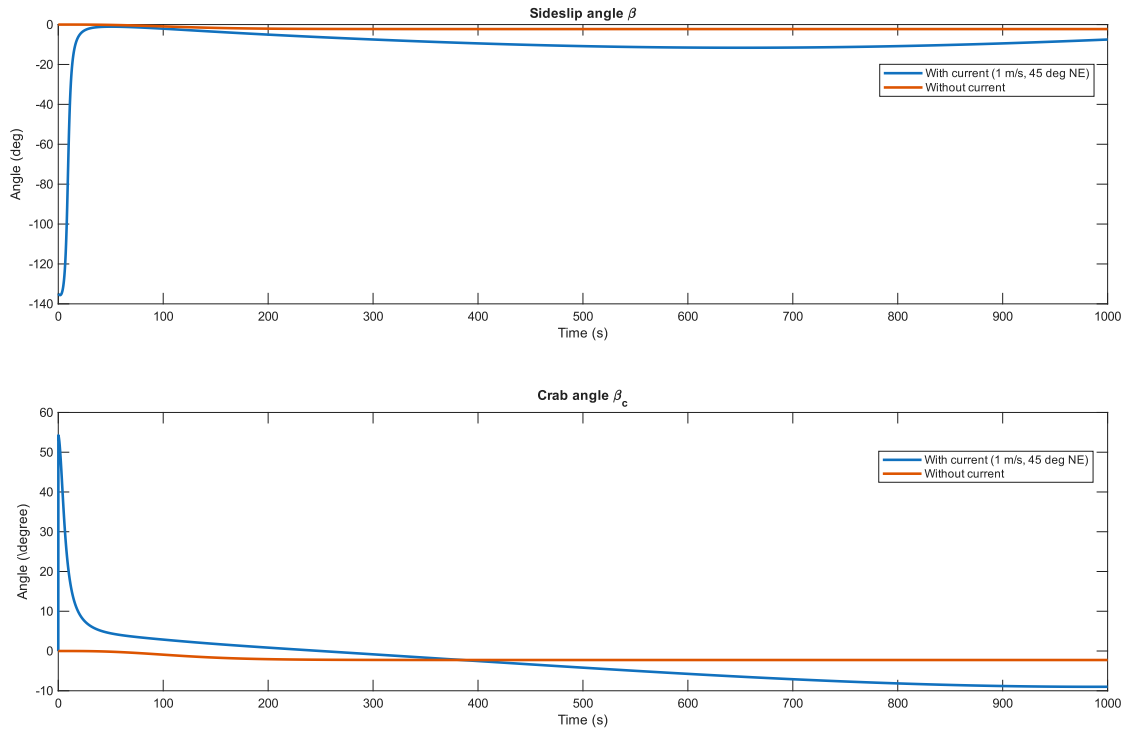


Figure 1: Sideslip versus crab angle, with and without current

Without currents, deviations from zero are mainly caused by the inertial elements and forces from the ship itself. A heavier ship will require more time before its course angle matches its heading angle.

With currents included, we see an initial sideslip angle of -145° , which can be counterintuitive at first. Since the ship starts out at rest with a heading angle of $\psi = 0^\circ$, the ocean current is the only contributing factor to the relative velocity. The current velocity in NED becomes:

$$\nu_c^b = \nu_c = \begin{bmatrix} V_c \cdot \cos 45^\circ \\ V_c \cdot \sin 45^\circ \end{bmatrix} = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix} \quad (1)$$

and the relative velocity becomes the negative of ν_c :

$$\nu_r = \nu - \nu_c = \begin{bmatrix} u_r \\ v_r \end{bmatrix} = \begin{bmatrix} -0.707 \\ -0.707 \end{bmatrix}, \quad (2)$$

giving us the initial sideslip angle:

$$\beta = \text{atan2}(v_r, u_r) = \text{atan2}(-0.707, -0.707) = -135^\circ. \quad (3)$$

As for the crab angle $\beta_c = \chi - \psi$, we observe how the vessel gradually gets rotated, increasing the difference between course angle χ and the ship's heading angle ψ .

1c Adding wind moments

```
Vw = 10; % wind speed (m/s)
beta_Vw = deg2rad(135); % wind speed direction
rho_a = 1.247; % air density
c_y = 0.95; % sway wind-force coefficient
c_n = 0.15; % Yaw wind-moment coefficient
L = 161; % Length of ship
A_Lw = 10*L; % Area of side view above water.

Ywind = 0.5*rho_a*Vw^2*c_y*A_Lw; % Equation from Fossen ch 10.1
Nwind = 0.5*rho_a*Vw^2*c_n*A_Lw*L; % Equation from Fossen ch 10.1
tau_wind = [0 Ywind Nwind]';
```

2 Heading Autopilot

2a Linearizing Coriolis forces

From the previous assignment, we have

$$C_{RB}(\nu) = \begin{bmatrix} 0 & 0 & -m(v + x_g r) \\ 0 & 0 & m u \\ m(v + x_g r) & -m u & 0 \end{bmatrix}, \quad C_A(\nu_r) = \begin{bmatrix} 0 & 0 & Y_{\dot{v}} v_r + Y_{\dot{r}} r \\ 0 & 0 & -X_{\dot{u}} u_r \\ -Y_{\dot{v}} v_r - Y_{\dot{r}} r & X_{\dot{u}} u_r & 0 \end{bmatrix}$$

Linearizing (setting $v = 0$, $r = 0$ and $u = u_d$) gives

$$C_{RB}^* = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & m u_d \\ 0 & -m u_d & 0 \end{bmatrix}, \quad C_A^* = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -X_{\dot{u}} u_d \\ 0 & X_{\dot{u}} u_d & 0 \end{bmatrix}$$

2b Transfer function

The sway–yaw subsystem is defined in (eq. 6.138 in (Fossen, 2021)) as

$$M \dot{x} + N x = b^* \delta, \quad x = \begin{bmatrix} v_r \\ r \end{bmatrix},$$

where

$$M = M_{RB} + M_A, \quad N = C_{RB}^* + C_A^* + D, \quad b^* = 2u_d \begin{bmatrix} -Y_{\delta} \\ -N_{\delta} \end{bmatrix}$$

$\dot{x}$ can be solved by first isolating $M\dot{x}$:

$$M \dot{x} = -N x + b^* \delta.$$

Then multiply by M^{-1}

$$\dot{x} = -M^{-1}N x + M^{-1}b^* \delta.$$

That gives us the state space model:

$$\dot{x} = Ax + B \delta, \quad A := -M^{-1}N, \quad B := M^{-1}b^*$$

Using the parameters from `ship.m` and linearizing at $U_d = 7$ m/s, the transfer function from rudder angle to yaw rate is

$$G_{r\delta}(s) = \frac{r(s)}{\delta(s)} = \frac{8.638 \times 10^{-5} s - 2.404 \times 10^{-6}}{s^2 + 0.1506 s + 0.02052}.$$

2c Nomoto model

The task asks for a first–order Nomoto model while we have a second–order system. It is possible to approximate a second–order Nomoto model with a first–order Nomoto model.

First, our system can be rearranged into the second–order Nomoto model by matching coefficients. The second–order Nomoto model is

$$\frac{r}{\delta}(s) = \frac{K(T_3 s + 1)}{(T_1 s + 1)(T_2 s + 1)} = \frac{K T_3 s + K}{T_1 T_2 s^2 + (T_1 + T_2) s + 1} \quad (4)$$

Divide by T_1T_2 to make the s^2 coefficient equal to 1:

$$\frac{r}{\delta}(s) = \frac{\frac{KT_3}{T_1T_2}s + \frac{K}{T_1T_2}}{s^2 + \frac{T_1+T_2}{T_1T_2}s + \frac{1}{T_1T_2}} \quad (5)$$

Matching the coefficients to the model from Task 2b we get the equations:

$$\frac{KT_3}{T_1T_2} = 8.638 \times 10^{-5}, \quad (6)$$

$$\frac{K}{T_1T_2} = -2.404 \times 10^{-6}, \quad (7)$$

$$\frac{T_1+T_2}{T_1T_2} = 0.1506, \quad (8)$$

$$\frac{1}{T_1T_2} = 0.02052 \quad (9)$$

Solving this gives us:

$$K = \boxed{-1.17154 \times 10^{-4}}$$

$$T_3 = -35.9318 \text{ s}$$

$$T_1 + T_2 = 7.33918$$

The first order Nomoto approximation uses:

$$T = T_1 + T_2 - T_3 = \boxed{43.27096 \text{ s}}$$

This gives us the model:

$$\boxed{\frac{r}{\delta}(s) \approx \frac{K}{Ts + 1} = \frac{-1.17154 \times 10^{-4}}{43.27096s + 1}} \quad (10)$$

2d PID Controller

Designing the PID controller using pole placement. Consider the equation of motion of a system.

$$m\ddot{x} + d\dot{x} + kx = \tau \quad (11)$$

The control law of τ is the PID controller.

$$\tau = kx_d - \left(K_p\tilde{x} + K_d\dot{\tilde{x}} + K_i \int_0^t \tilde{x}(\tau) d\tau \right) \quad (12)$$

Where $K_d > 0$, $K_p > 0$ & $K_i > 0$. Simplifying the expression by setting $K_i = 0$. Then insert these terms in Equation 11 to get the following expression.

$$m\ddot{x} + (d + K_d)\dot{x} + (k + K_p)x = 0 \quad (13)$$

Then we use the following relations of critical damping ζ and natural frequency ω_n .

$$\zeta = \frac{d + K_d}{2m\omega_n}, \quad \omega_n = \sqrt{\frac{k + K_p}{m}}$$

By specifying ω_n & ζ , the PD-gains can be computed from these relations.

$$K_p = m\omega_n^2 - k, \quad K_d = 2\zeta\omega_n m - d$$

As for the integral gain K_i , we use the rule of thumb from Fossen by setting it 10 times slower than the natural frequency.

$$K_i = \frac{\omega_n}{10} K_p = \frac{\omega_n}{10} (m\omega_n^2 - k)$$

The natural frequency is estimated by relating it to bandwidth frequency ω_b . Bandwidth frequency is the frequency at which the loop transfer function $l(s) = h(s) \cdot 1$ satisfies,

$$|l(j\omega)|_{\omega=\omega_b} = \frac{\sqrt{2}}{2}, \quad \text{or} \quad 20\log|l(j\omega)|_{\omega=\omega_b} = -3 \text{ dB}$$

The bandwidth frequency is defined in the Fossen book to be the following expression from a second order system.

$$\omega_b = \omega_n \sqrt{1 - 2\zeta^2 + \sqrt{4\zeta^4 - 4\zeta^2 + 2}}$$

For a critically damped system $\zeta = 1.0$, the expression is reduced.

$$\omega_b = \omega_n \sqrt{\sqrt{2} - 1} \approx 0.64\omega_n$$

In this task, the bandwidth frequency is $\omega_b = 0.06$ rad/s and relative damping is $\zeta = 1.0$.

Code implementation of reference model.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Part 2, 2d) Add a reference model here
% Define it as a function
% check eq. (15.143) in (Fossen, 2021) for help
%
% The result should look like this:
% xd_dot = ref_model(xd, psi_ref(i));
% psi_d = xd(1);
% r_d = xd(2);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
wb_ref = 0.03;
zeta_ref = 1.0;
xd = [0, 0, 0];
xd_dot = ref_model(xd, psi_ref, wb_ref, zeta_ref);
xd = xd + h*xd_dot;
psi_d = xd(1);
r_d = xd(2);
r_d_dot = xd_dot(2);
u_d = U_ref;

function xd_dot = ref_model(xd, psi_ref)
% REF_MODEL - 3rd order reference model from Fossen (2021, eq. 15.143)
%
% State vector:
%   xd = [psi_d; r_d; v_d]
%
% Inputs:
%   psi_ref - reference heading (rad)
%
% Parameters:
%   wb_ref - natural frequency (rad/s)
```

```

% zeta_ref - damping ratio
%
% Output:
% xd_dot = [psi_d_dot; r_d_dot; v_d_dot]

% --- Model parameters ---
wb_ref = 0.03; % natural frequency (rad/s)
zeta_ref = 1.0; % damping ratio

% --- State variables ---
psi_d = xd(1);
r_d = xd(2);
v_d = xd(3);

% --- Coefficients from Fossen (15.143) ---
a2 = (2*zeta_ref + 1) * wb_ref;
a1 = (1 + 2*zeta_ref) * wb_ref^2;
a0 = wb_ref^3;
g = wb_ref^3;

% --- State derivatives ---
psi_d_dot = r_d;
r_d_dot = v_d;
v_d_dot = -a2*v_d - a1*r_d - a0*psi_d + g*psi_ref;

% --- Output ---
xd_dot = [psi_d_dot; r_d_dot; v_d_dot];
end

```

Code implementation of heading control.

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Part 2, 2d) Add the heading controller here
% Define it as a function
%
% The result should look like this:
% delta_c = PID_heading(e_psi,e_r,e_int);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
wb = 0.06; zeta = 1.0; alpha = 1.0;
T_nom = 43.27096; K_nom = -1.17154e-4;
a2 = (2*zeta + alpha)*wb; a1 = (1 + 2*zeta)*wb^2; a0 = alpha*wb^3;

kp = (T_nom*a1)/K_nom;
kd = (T_nom*a2 - 1)/K_nom;
ki = (T_nom*a0)/K_nom;

delta_max = deg2rad(25);

P.kp = kp; P.kd = kd; P.ki = ki;
P.dm = delta_max; P.h = h; P.Kaw = 5;
assignin('base','PIDH_PERSIST',P);

e_int = 0;

psi = x(6);
r = x(3);
e_psi = ssa(psi_d-psi);
e_r = r_d - r;

```

```

[delta_c, e_int] = PID_heading(e_psi, e_r, e_int);

function [delta_c, e_int] = PID_heading(e_psi, e_r, e_int)
% PID heading controller with anti-windup
P = evalin('base', 'PIDH_PERSIST');

% PID law (continuous approximation)
delta_c = -(P.kp*e_psi + P.kd*e_r + P.ki*e_int);

% Update integral term
e_int = e_int + P.h*e_psi;

% Store updated integral in workspace (persistent state)
P.e_int = e_int;
assignin('base', 'PIDH_PERSIST', P);
end

```

2e Controller performance

The reference heading was defined as a time-varying signal. From $t = 0$ to $t = 500$ seconds, the reference heading was set to 10° , and from $t = 500$ to $t = 1000$ seconds, it was changed to -20° .

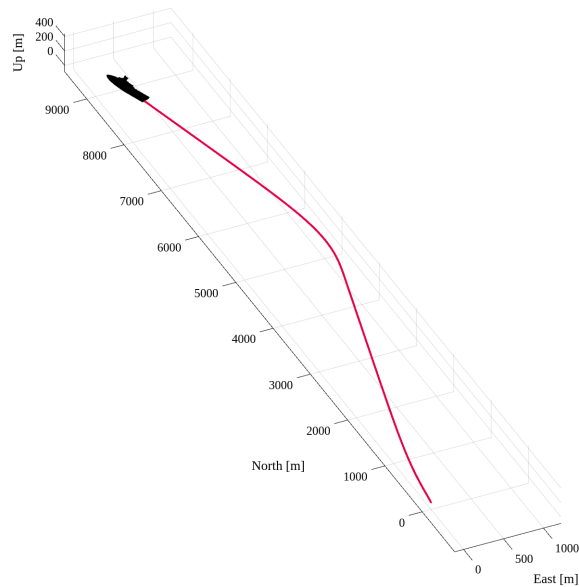


Figure 2: Resulting path.

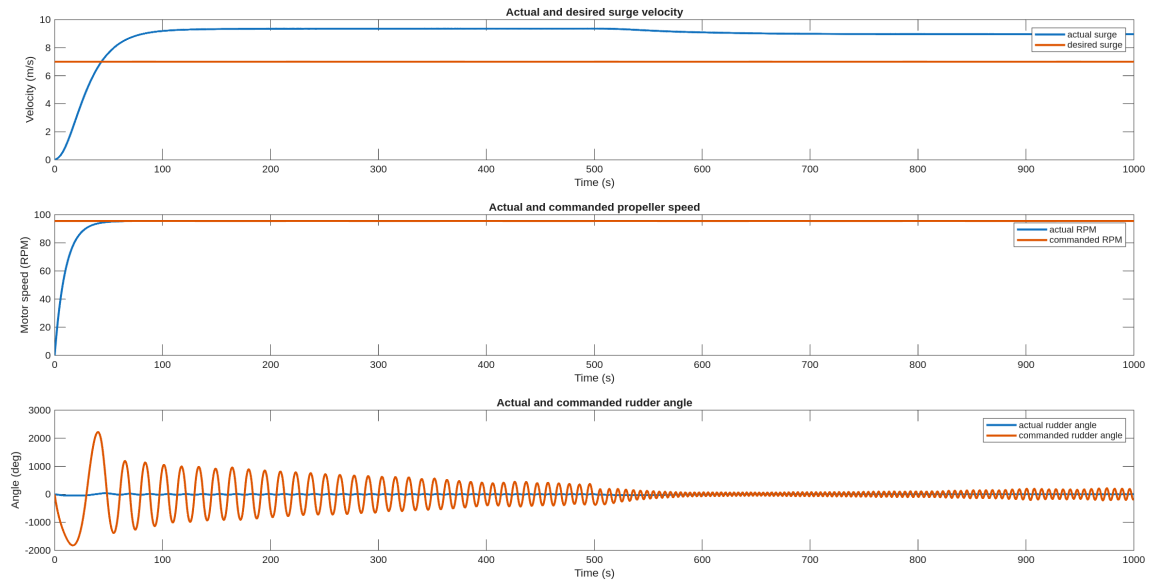


Figure 3: Resulting surge velocity, propeller speed, and rudder angle.

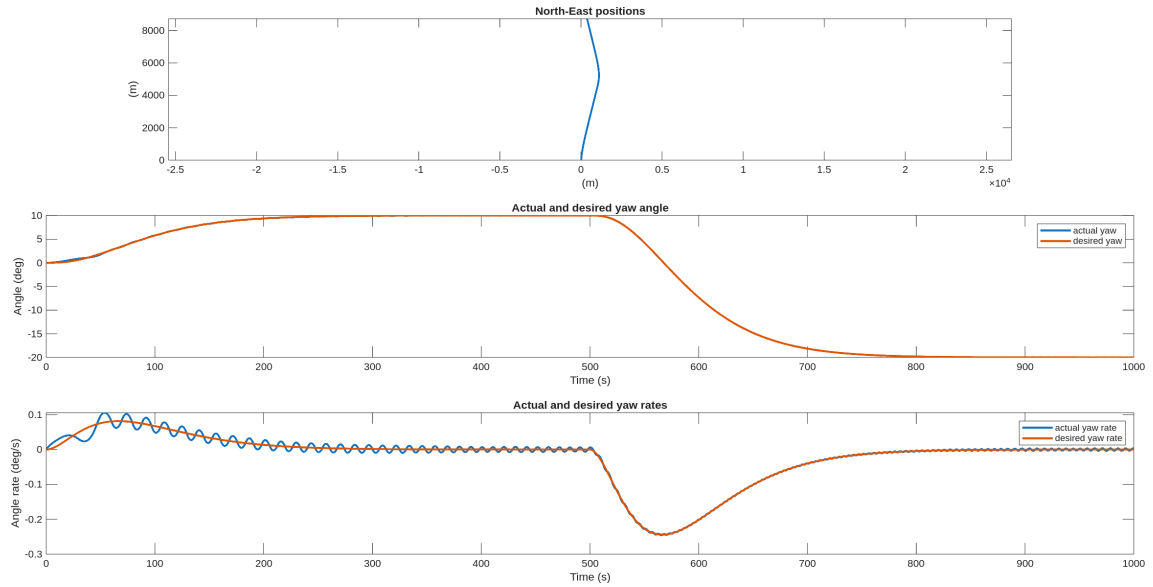


Figure 4: Resulting North-East path, yaw angle, and yaw rate.

As shown in Figure 2 and Figure 4, the controller successfully tracks the time-varying heading reference. However, Figure 3 reveals that the commanded rudder angle frequently exceeds the actuator limits, which leads to integrator windup. To mitigate this, an anti-windup plus a saturation mechanism was implemented by clamping the control input to $\pm 40^\circ$. The updated PID controller with conditional integration is shown below and was used in all subsequent simulations.

```
function [delta_c, e_int] = PID_heading(e_psi, e_r, e_int)
    % PID heading controller with anti-windup
    P = evalin('base', 'PIDH_PERSIST');

    % Compute the unsaturated control signal
    delta_unsat = -(P.kp*e_psi + P.kd*e_r + P.ki*e_int);

    delta_c = delta_unsat;
```

```
e_int = e_int + P.h*e_r;

% Saturated control law and integrator anti-windup
if delta_unsat > P.delta_max
    delta_c = P.delta_max; % Saturation
    e_int = e_int - (P.h/P.ki) * (delta_c - delta_unsat); % Anti-windup

elseif delta_unsat < -P.delta_max
    delta_c = -P.delta_max; % Saturation
    e_int = e_int - (P.h/P.ki) * (delta_c - delta_unsat); % Anti-windup
end

% Save state
P.e_int = e_int;
assignin('base', 'PIDH_PERSIST', P);
end
```

3 Propeller Revolution and Speed Control

3a Propeller thrust and torque coefficients

The propeller coefficients can be calculated using the wageningen function:

```
Ja = 0;           % Advance number
PD = 1.5;         % Pitch to diameter ratio
AEAO = 0.65;      % Blade area ratio
z = 4             % nr of blades

[KT, KQ] = wageningen(Ja,PD,AEAO,z)
```

This results in $K_T = 0.6367$ and $K_Q = 0.1390$.

3b Torque to speed

The relation between desired propeller torque Q_d and desired shaft speed η_d (in rps) can be expressed with:

$$Q_d = \rho D^5 K_Q(0) |n_d| n_d$$

3c Propulsion system code

The trust and torque can be computed using:

```
thr = rho * Dia^4 * KT * abs(n) * n; % thrust command (N)
Q = rho * Dia^5 * KQ * abs(n) * n;
```

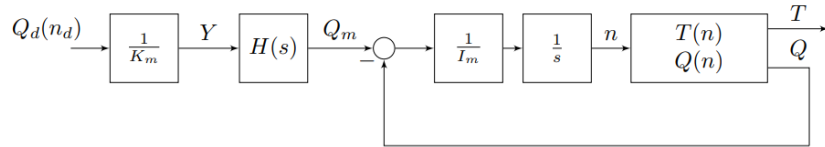


Figure 5: Propulsion system from the assignment.

The transfer function $H(s)$ is defined as:

$$H(s) = \frac{Q_m(s)}{Y(s)} = \frac{K_m}{T_m s + 1} e^{-\tau s}$$

It must be converted to time domain, and can be done using the inverse Laplace. The part $e^{-\tau s}$ disappears because $\tau = 0$. From Figure 5 we have $Y = Q_d/K_m$. Rearranging, we get:

$$Q_m(s) = \frac{K_m}{T_m s + 1} Y(s) = \frac{K_m}{T_m s + 1} \frac{Q_d(s)}{K_m} = \frac{1}{T_m s + 1} Q_d(s)$$

Multiplying both sides by the denominator and rearranging gives us:

$$(T_m s + 1)Q_m(s) = Q_d(s) \Rightarrow T_m s Q_m(s) + Q_m(s) = Q_d(s)$$

Taking the inverse Laplace transform gives the time-domain expression:

$$T_m \dot{Q}_m(t) + Q_m(t) = Q_d(t),$$

which can be rearranged into the ordinary differential equation

$$\dot{Q}_m = \frac{Q_d - Q_m}{T_m}$$

The shaft dynamics can be derived directly from Figure 5:

$$\dot{n} = \frac{Q_m - Q}{I_m}.$$

The implementation corresponds to the code:

```
Tm = 10; % s
Im = 100000; %kg m^2

Qd = rho * Dia^5 * KQ * abs(n_c) * n_c;
Qm_dot = (Qd-Qm)/Tm;
n_dot = (Qm-Q) * 1/Im;
```

In addition, Q_m and $\dot{Q}_m$ was respectively added to the state variable $\mathbf{x}$ and $\dot{\mathbf{x}}$.

3d Cruise speed

The steady-state / cruise speed relationship

$$U = \frac{(t-1)T}{X_u} \tag{14}$$

holds when the surge dynamics (relationship between thrust $\mathbf{T}$ and surge damping $\mathbf{X}_u$) are 1st-order, linear and with constant thrust ($m\dot{u} + X_u u = (t-1)T$, $\dot{u} = 0$).

3e Desired thrust, speed & simulation

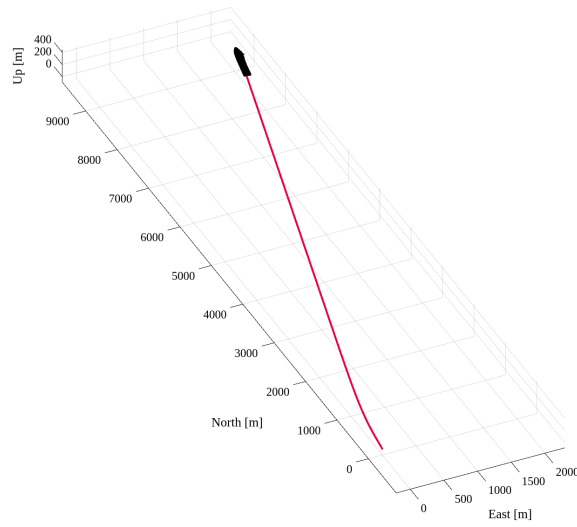


Figure 6: Resulting path.

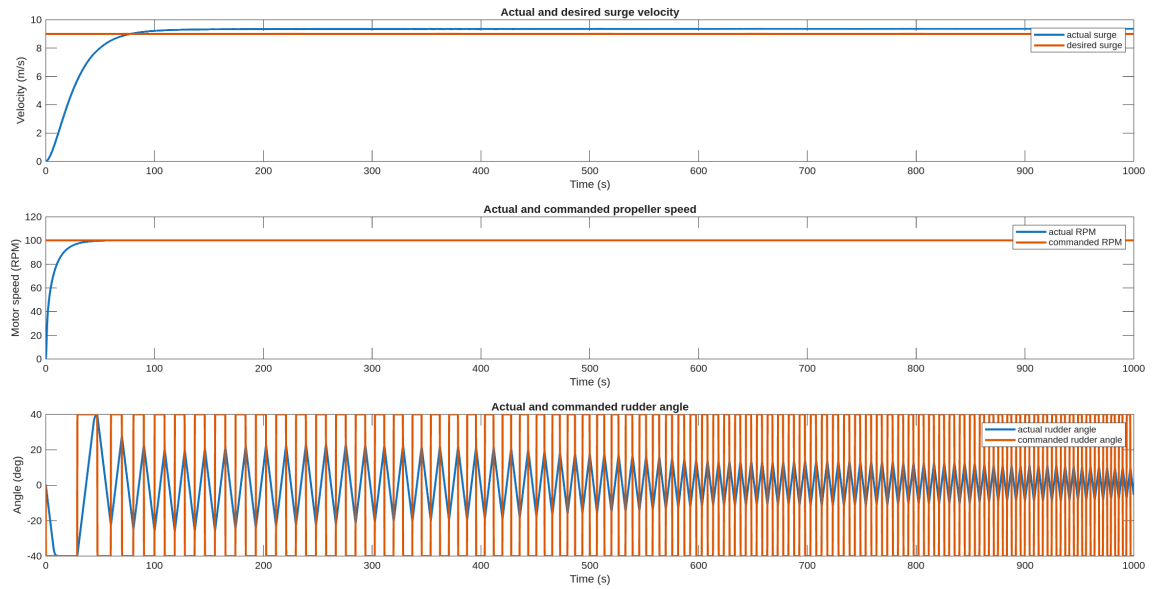


Figure 7: Resulting surge velocity, propeller speed, and rudder angle.

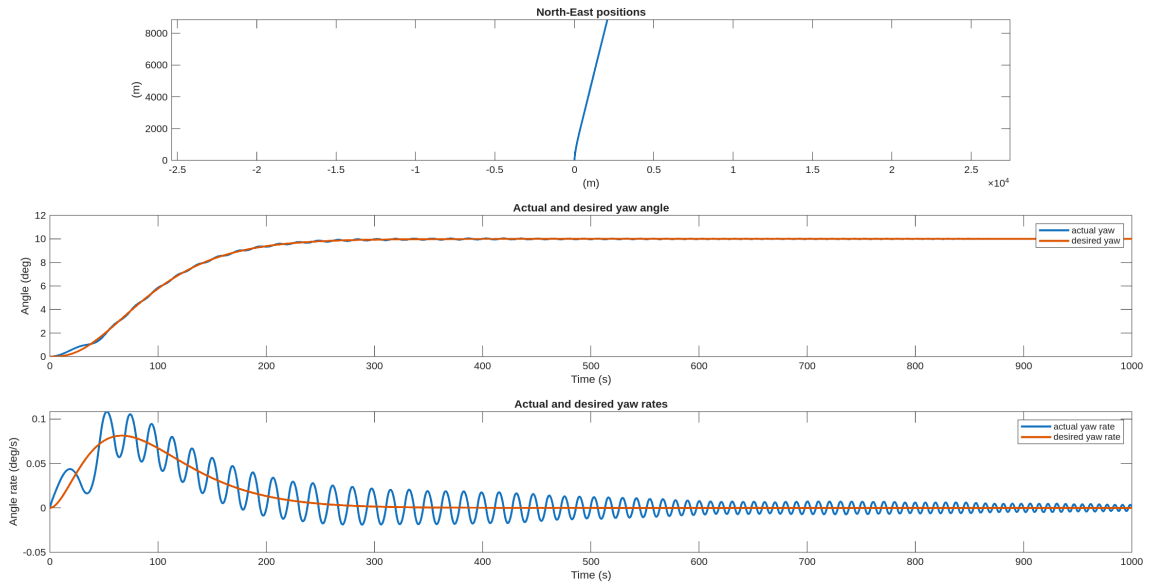


Figure 8: Resulting North-East path, yaw angle, and yaw rate.

Since this is open-loop speed control, any disturbance will add to a steady-state offset to the surge speed. The controller does not account for the external wind and current forces, which combined contribute to a slight positive offset in this case, giving the vessel an extra push.

3f Optional Improvements

The steady-state offset in surge speed as observed in 3e could be avoided by simply adding an integrator to the speed error in the controller. An even more robust approach would be to add a feedforward thrust that implement the wind and current forces in the control law, then control n_c based on that. The feedforward method accounts for the external forces before the error is even observed.